

DATA 442/621: Deep Learning and Neural Networks

Assignment 0: Getting Started

Assigned: Aug 26, 2026; Due: Sep 11, 2026

Instructions

The objective of this assignment is to make sure that you are able to access and use a PyTorch computing environment. If you have a SciClone account, you can access the JupyterHub following the instructions here: <https://www.wm.edu/offices/it/services/researchcomputing/jupyter/>. If you do not have a SciClone account, you may register for one here: <https://hpc.wm.edu/acctreq/>. If you intend to work using a personal laptop, please ensure it has a dedicated graphics card compatible with PyTorch (see here: <https://pytorch.org/get-started/locally/>).

This assignment will be graded out of 10 points. It will contribute bonus points to your overall assignments grade.

Please submit the following:

1. A Jupyter notebook executing the requested code cells
2. A text file answering each **Question**

Part 1: Installing and importing torch (2 points)

If you are using a personal machine, install `torch` into your Python environment following the instructions at <https://pytorch.org/get-started/locally/>. Alternatively, in a Jupyter-like environment, you may run `!pip install torch` in a separate code cell.

Question 1: Execute the following snippet in your Jupyter notebook. What version of torch is installed in your environment?

```
import torch
print(f"Using PyTorch version {torch.__version__}")
```

Part 2: Hardware acceleration (2 points)

Question 2: In the same Jupyter notebook, execute the following snippet. What device(s) can torch use in your environment?

```
if torch.cuda.is_available():
    device = torch.device('cuda')
    print(f'GPU is available, using {device}')
elif torch.mps.is_available():
    device = torch.device('mps')
    print(f'Apple Metal Performance Shaders framework is available, using {device}')
else:
    device = torch.device('cpu')
    print(f'Hardware acceleration not available, using {device}')
```

Question 3: If you are only able to use `cpu`, please note whether you plan to use SciClone or Google Colab or a similar service for hardware-accelerated programming in this course. Otherwise, tell me about which ride at Busch Gardens scares you the most.

Part 3: Some manipulations (6 points)

This section has some code snippets to check that some essential PyTorch operations work, including variable initialization and gradient tracking.

Question 4: Execute the following code snippet in your Jupyter notebook. What is the difference between the value of dL/dw and what it “should” be, according to the notebook output?

```
kwargs = {'dtype': torch.float, 'device': device} # For tensor initialization

w = torch.tensor(1., requires_grad=True, **kwargs)
b = torch.tensor(0., requires_grad=True, **kwargs)

x = torch.tensor([0.5], **kwargs)
y = torch.tensor([6.8], **kwargs)

z = w * x + b
loss = (y - z).pow(2).sum()
loss.backward()
print(f'The squared error is {loss.item()}')
print('-----')
print(f'dL/dw = {w.grad}, should be {2*x*(z - y)}')
print(f'dL/db = {b.grad}, should be {2*(z - y)}')
```

Question 5: Execute the following code snippet in your Jupyter notebook. What is the “new” squared error? How much lower is it than the original squared error in the previous cell?

```
alpha = 0.01 # Learning rate
with torch.no_grad():
    w = w - alpha * w.grad
    b = b - alpha * b.grad

print(f'The new value of w is {w.item()}')
print(f'The new value of b is {b.item()}')
print('-----')
z = w * x + b
loss = (y - z).pow(2).sum()

print(f'The new squared error is {loss.item()}')
```

Question 6: Execute the code snippet below and paste the output into your submitted text file.

```
import numpy as np

w_numpy = w.numpy()
w_item = w.item()
print(f"The difference between .numpy() and .item() is {w_numpy - w_item}")
```